```
#!/usr/bin/env bash
# harden_v1_1.sh — apply to a repo bootstrapped with bootstrap_notices.sh. Run from repo root.
# Validates every anchor in every file first; writes nothing unless all match (edited since v1 =>
abort).
set -euo pipefail

for f in scripts/generate_notices.py tests/test_generate_notices.py data/sample_debts.json; do
  [ -f "$f" ] || { echo "missing $f (run bootstrap_notices.sh first)" >&2; exit 2; }
done
ts=$(date +%s)
for f in scripts/generate_notices.py tests/test_generate_notices.py data/sample_debts.json; do
  cp -p "$f" "$f.bak.$ts"
done

python3 - <<'PYEOF'
import json
from pathlib import Path

PENDING = []


def patch(path, pairs):
    s = Path(path).read_text(encoding="utf-8")
    for i, (old, new) in enumerate(pairs, 1):
        n = s.count(old)
        if n != 1:
            raise SystemExit(f"{path}: anchor {i} matched {n}x; nothing written")
        s = s.replace(old, new)
    PENDING.append((Path(path), s))

GEN = [
# 1. imports
("import argparse\nimport hashlib\n", "import argparse\nimport contextlib\nimport hashlib\n"),

# 2. strict date-only parsing (timestamps shift the calendar day in local time)
('''def parse_date(value, field: str = "date") -> date:
    if isinstance(value, date):
        return value
    try:
        return date.fromisoformat(str(value)[:10])
    except ValueError:
        raise RecordError(f"{field}: invalid date {value!r}") from None
''',
'''DATE_RE = re.compile(r"^\\d{4}-\\d{2}-\\d{2}$")


def parse_date(value, field: str = "date") -> date:
    """Date-only ISO strings. Timestamps are rejected: truncating '...T00:00:00Z'
    would shift the calendar day for any local timezone behind UTC."""
    if type(value) is date:
        return value
```

```
    try:
        if not DATE_RE.match(str(value)):
            raise ValueError(value)
        return date.fromisoformat(str(value))
    except ValueError:
        raise RecordError(
            f"{field}: invalid date {value!r} (expected YYYY-MM-DD, date only)"
        ) from None
'''),

# 3. human verification gate
("def _wrap(*paragraphs: str) -> str:",
'''def check_verified(record: dict, today: date) -> None:
    """A person must vouch for the source records before any document is generated."""
    require(record, ("verified_by", "verified_on"))
    if parse_date(record["verified_on"], "verified_on") > today:
        raise RecordError("verified_on is in the future")


def _wrap(*paragraphs: str) -> str:'''),

# 4. atomic private writes, private dir, safe manifest load
('''def _write_private(path: Path, text: str) -> None:
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
    with os.fdopen(fd, "w", encoding="utf-8") as fh:
        fh.write(text)
    os.chmod(path, 0o600)
''',
'''def _write_private(path: Path, text: str) -> None:
    """Write 0600 via temp file + rename: no partial files, no inherited loose mode."""
    tmp = path.with_name(f".{path.name}.{os.getpid()}.tmp")
    fd = os.open(tmp, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as fh:
            fh.write(text)
            fh.flush()
            os.fsync(fh.fileno())
        os.replace(tmp, path)
    except BaseException:
        with contextlib.suppress(FileNotFoundError):
            os.unlink(tmp)
        raise


def _ensure_private_dir(path: Path) -> None:
    created = not path.exists()
    path.mkdir(parents=True, exist_ok=True)
    if created:
        os.chmod(path, 0o700)


def _load_manifest(path: Path) -> dict:
    if not path.is_file():
        return {}
    try:
```

```
        return {e["file"]: e for e in json.loads(path.read_text(encoding="utf-8"))}
    except (json.JSONDecodeError, KeyError, TypeError):
        raise RuntimeError(f"corrupt manifest, refusing to overwrite: {path}") from None
'''),

# 5. run(): preload manifest (fail before writing anything)
('''    result = {"written": [], "failed": [], "skipped": 0}
    manifest = []
''',
'''    result = {"written": [], "failed": [], "skipped": 0}
    manifest = []
    manifest_path = out_dir / "manifest.json"
    prior = {} if dry_run else _load_manifest(manifest_path)
'''),

# 6. run(): verification gate before generation
('''            if not acct:
                raise RecordError("missing field(s): account_id")
            text = generate(rec, today)
''',
'''            if not acct:
                raise RecordError("missing field(s): account_id")
            check_verified(rec, today)
            text = generate(rec, today)
'''),

# 7. manifest records who verified
('''            "sha256": hashlib.sha256(text.encode("utf-8")).hexdigest(),
        })
''',
'''            "sha256": hashlib.sha256(text.encode("utf-8")).hexdigest(),
            "verified_by": str(rec["verified_by"]),
            "verified_on": str(rec["verified_on"]),
        })
'''),

# 8. private output dir
('''            out_dir.mkdir(parents=True, exist_ok=True)
            _write_private(out_dir / name, text)
''',
'''            _ensure_private_dir(out_dir)
            _write_private(out_dir / name, text)
'''),

# 9. manifest merges across runs instead of clobbering
('''    if manifest and not dry_run:
        _write_private(out_dir / "manifest.json",
                       json.dumps(manifest, indent=2, sort_keys=True) + "\\n")
''',
'''    if manifest and not dry_run:
        merged = dict(prior)
        merged.update({e["file"]: e for e in manifest})
        _write_private(
            manifest_path,
            json.dumps(sorted(merged.values(), key=lambda e: e["file"]),
```

```
                        indent=2, sort_keys=True) + "\\n",
        )
'''),

# 10. main(): corrupt manifest -> exit 2
('''    res = run(debts, Path(args.output), today, args.dry_run)
''',
'''    try:
        res = run(debts, Path(args.output), today, args.dry_run)
    except RuntimeError as exc:
        print(f"[ERROR] {exc}", file=sys.stderr)
        return 2
'''),
]
patch("scripts/generate_notices.py", GEN)

TESTS = [
('''        "consumer_address": "1 Test Way, Harrisburg, PA 17101",
    }
''',
'''        "consumer_address": "1 Test Way, Harrisburg, PA 17101",
        "verified_by": "J. Reviewer",
        "verified_on": "2026-11-01",
    }
'''),
('''        "eca_initiation_date": "2026-04-01",  # 71 days after billing
    }
''',
'''        "eca_initiation_date": "2026-04-01",  # 71 days after billing
        "verified_by": "J. Reviewer",
        "verified_on": "2026-11-01",
    }
'''),
("class Cli(Base):",
'''class Hardening(Base):
    def test_timestamps_rejected(self):
        for bad in ("2026-11-05T00:00:00Z", "2026-11-05 00:00:00", "11/05/2026"):
            with self.assertRaisesRegex(gn.RecordError, "date only"):
                gn.parse_date(bad, "x")

    def test_unverified_record_never_generates(self):
        for rec in (sol_record(verified_by=""), sol_record(verified_on=None)):
            buf = io.StringIO()
            with contextlib.redirect_stderr(buf):
                res = gn.run([rec], self.dir, TODAY)
            self.assertEqual((len(res["written"]), len(res["failed"])), (0, 1))
            self.assertIn("verified", buf.getvalue())
        self.assertEqual(list(self.dir.iterdir()), [])

    def test_future_verification_rejected(self):
        res = quiet(gn.run, [sol_record(verified_on="2026-12-01")], self.dir, TODAY)
        self.assertEqual(len(res["failed"]), 1)

    def test_atomic_private_write_fixes_loose_mode_and_leaves_no_temp(self):
        target = self.dir / "x.txt"
```

```
        target.write_text("old")
        os.chmod(target, 0o644)
        gn._write_private(target, "new")
        self.assertEqual(target.read_text(), "new")
        self.assertEqual(stat.S_IMODE(target.stat().st_mode), 0o600)
        self.assertEqual([p.name for p in self.dir.iterdir()], ["x.txt"])

    def test_new_output_dir_is_0700(self):
        out = self.dir / "a" / "b"
        gn.run([sol_record()], out, TODAY)
        self.assertEqual(stat.S_IMODE(out.stat().st_mode), 0o700)

    def test_failed_write_cleans_up_temp(self):
        with mock.patch("generate_notices.os.replace", side_effect=OSError("boom")):
            with self.assertRaises(OSError):
                gn._write_private(self.dir / "y.txt", "data")
        self.assertEqual(list(self.dir.iterdir()), [])

    def test_missing_account_id_fails_under_unknown_tag(self):
        buf = io.StringIO()
        with contextlib.redirect_stderr(buf):
            res = gn.run([sol_record(account_id="")], self.dir, TODAY)
        self.assertEqual(len(res["failed"]), 1)
        self.assertIn("ID-unknown", buf.getvalue())

    def test_negative_amount_and_date_objects(self):
        with self.assertRaisesRegex(gn.RecordError, "amount"):
            gn.generate_pa_sol_notice(sol_record(amount="-5"), TODAY)
        self.assertEqual(gn.parse_date(date(2026, 1, 1)), date(2026, 1, 1))

    def test_manifest_merges_across_runs(self):
        gn.run([sol_record()], self.dir, TODAY)
        gn.run([sol_record(account_id="ACC-2222")], self.dir, TODAY)
        gn.run([sol_record()], self.dir, TODAY)  # rerun: same file, no duplicate
        manifest = json.loads((self.dir / "manifest.json").read_text())
        self.assertEqual(len(manifest), 2)
        self.assertTrue(all(m["verified_by"] == "J. Reviewer" for m in manifest))

    def test_corrupt_manifest_is_never_overwritten(self):
        (self.dir / "manifest.json").write_text("{broken")
        src = self.dir / "d.json"
        src.write_text(json.dumps([sol_record()]))
        args = ["--input", str(src), "--output", str(self.dir), "--today", "2026-11-05"]
        self.assertEqual(quiet(gn.main, args), 2)
        self.assertEqual((self.dir / "manifest.json").read_text(), "{broken")
        self.assertEqual(list(self.dir.glob("*.txt")), [])


class Cli(Base):'''),
]
patch("tests/test_generate_notices.py", TESTS)

# sample data gets verification stamps (synthetic)
sample = Path("data/sample_debts.json")
rows = json.loads(sample.read_text(encoding="utf-8"))
```

```
for r in rows:
    r.setdefault("verified_by", "SAMPLE REVIEWER")
    r.setdefault("verified_on", "2026-11-01")
PENDING.append((sample, json.dumps(rows, indent=2, ensure_ascii=False) + "\n"))

for path, text in PENDING:  # all anchors matched: write
    path.write_text(text, encoding="utf-8")
print("patched")
PYEOF

export HMAC_SECRET="${HMAC_SECRET:-$(python3 -c 'import secrets;
print(secrets.token_urlsafe(48))')}"
python3 -m unittest discover -s tests
python3 scripts/generate_notices.py --input data/sample_debts.json --dry-run --today 2026-11-05
echo "v1.1 OK. Backups: *.bak.$ts"
```